

Where Does Meta-Learning Help a Quadrotor?

A Simulation Study of Seven Algorithms with Classical Control and Planning

Amal Mehta

TODO: affiliation

amal.mehta@gmail.com

Abstract—A quadrotor deployed outside the lab meets conditions it was not tuned for: extra payload, a worn motor, wind and command latency. Meta-reinforcement learning promises adaptation from a few flights, while classical stacks built on geometric control, adaptive control, occupancy mapping and A* planning are dependable but fixed. We ask where meta-learning helps when it is combined with such a stack. In a simulator parameterised after a Holybro X500 V2, we insert learning at three points (a residual on a geometric controller, the controller’s gains, and the local planner of a depth-camera, mapping and A* navigation stack) and compare seven algorithms (MAML, FOMAML, ANIL, Meta-SGD, Reptile, PEARL and RL²) with nominal, L1-adaptive, domain-randomised and fine-tuning baselines on held-out and out-of-distribution dynamics. A learned residual roughly halves tracking error relative to tuned L1 adaptation (7.3 vs 14.0 cm), and meta-learning adds little to it. Meta-learning pays off when it adapts the classical gains: Meta-SGD lowers error from 15.8 to 10.8 cm, 3.5 cm better than non-meta tuning on paired tasks. In navigation, gradient-based adaptation at the meta-trained step size degrades learned planners, and across 96 flights in unseen rooms the classical local planner is the most reliable (73% success, 7% collisions). Outside the training distribution every method fails, and learned residuals crash more often than the classical controller. We release the suite and recommend a platform and staged plan for real-world tests.

Index Terms—Aerial robotics, meta-learning, adaptive control, reinforcement learning, motion planning.

I. INTRODUCTION

Small aerial robots rarely fly the vehicle they were tuned on. A delivery adds payload, a propeller chips, the wind picks up and the companion computer adds latency. Classical control handles part of this: a geometric tracking controller [1] with integral action rejects slow disturbances, and L1 adaptive augmentation estimates and cancels a lumped disturbance online [2]. Learning-based adaptation promises more: meta-learning trains a model that adapts from a few samples of a new task [3], and learned adaptation has produced agile flight in strong winds [4] and single controllers for very different quadrotors [5], [6].

Two questions remain open. First, meta-reinforcement-learning algorithms are typically evaluated against each other on simulated robot benchmarks such as locomotion and manipulation suites [7], [8], rather than against the classical controllers and planners that would run in the same loop. Second, a deployed autonomy stack has several places where

learning could enter (the low-level controller, its gains, the local planner), and it is not clear which of them benefits from meta-learning.

We study both questions in one simulation suite. A batched quadrotor model parameterised after a Holybro X500 V2 runs a classical stack: a geometric controller with collective-thrust and body-rate (CTBR) output, a simulated RealSense D435i depth camera, a log-odds occupancy map and an A* planner (Fig. 1). Meta-learning plugs into three places: a residual on the controller, the controller’s gains, and the local planner. Each must adapt to a new task drawn from payload, motor wear, drag, wind, gusts and latency.

Our contributions are:

- an open simulation suite with three classical+meta combinations and a reproducible benchmark protocol (Secs. III–VI);
- a comparison of seven meta-learning algorithms with nominal, L1-adaptive, domain-randomised (DR) and fine-tuning baselines on held-out and out-of-distribution (OOD) tasks (Sec. VII);
- four findings: a learned residual beats tuned classical adaptation; meta-learning helps most when it adapts classical gains; gradient-based adaptation at the meta-trained step size degrades learned navigation planners, and the classical planner stays the most reliable on the full stack; and no method survives the OOD tasks;
- a hardware recommendation and staged sim-to-real plan for testing the stack (Sec. VIII).

Code, results and a replay of recorded flights are available at <https://github.com/amalmehta/DroneAutonomy>.

II. RELATED WORK

Meta-reinforcement learning. Gradient-based methods learn an initialisation from which a few policy-gradient steps adapt to a new task: MAML [3], its first-order approximation FOMAML, Reptile [9], ANIL, which adapts only the output layer [10], and Meta-SGD, which also learns per-parameter step sizes [11]. Context-based methods instead infer the task from experience: PEARL encodes recent transitions into a latent variable for an off-policy actor-critic [12], and RL² uses a recurrent policy whose state carries what it has learned across episodes [13], [14]. They are typically benchmarked against each other on simulated locomotion and manipulation [7], [8];

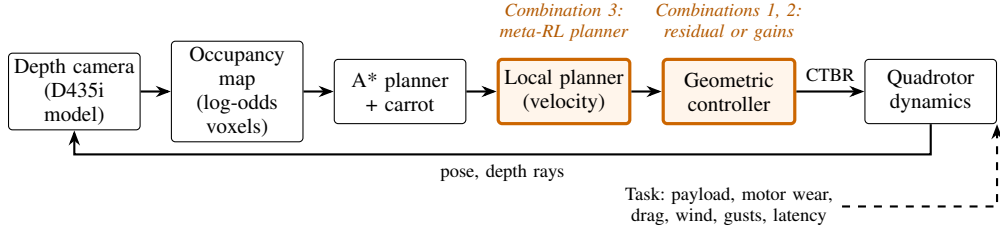


Fig. 1. Meta-learning plugs into three places of one classical stack. The local planner (Combination 3) turns depth and a carrot point on the A* path into a velocity command. The geometric controller turns that into collective thrust and body rates, with a learned residual (Combination 1) or meta-learned gains (Combination 2). The task perturbs the dynamics.

we compare them inside a classical flight stack and against classical adaptive control.

Learned adaptation for flight. Neural-Fly learns wind-invariant features offline and adapts a linear layer online [4]; adaptive-control-oriented meta-learning trains features for an adaptive law [15]; rapid motor adaptation infers latent dynamics from history [16], an idea carried to quadrotors of very different sizes [5]. Domain randomisation trains a single robust policy instead [17], [18], [19]. L1 adaptive augmentation of a geometric controller is a strong classical alternative [2]. Closest to this paper, MAVEN meta-trains a policy across varying quadrotor dynamics for agile manoeuvres [20], RAPTOR trains one adaptive policy that flies many different quadrotors [6], and multi-task RL shares one policy across quadrotor tasks [21]. RL has also been used to predict a quadrotor’s PID gains [22], the non-meta counterpart of our Combination 2. Each of these proposes one learner; we compare seven meta-learners against classical, L1 and domain-randomised baselines at three insertion points of the same stack.

Residual and hybrid control. Residual RL learns a correction on top of a fixed controller [23], [24]. Our Combinations 1 and 2 are residual and gain-tuning variants of this idea, trained with meta-learning.

Learned versus classical navigation. Classical stacks build a map, for example an OctoMap [25], and plan on it with search such as A* [26]. Learned policies can fly fast through clutter from depth alone [27], and RL can outperform optimal control in racing [28]. We keep the map and planner and ask whether a meta-learned local planner improves on a classical one. Open simulators and platforms for learning-based flight include gym-pybullet-drones [29] and Agilicious [30], and safe-control-gym benchmarks learning-based control, including quadrotors, against classical control [31].

III. PROBLEM FORMULATION

A task τ fixes physical parameters $\xi_\tau \in \Xi$ (payload, motor efficiencies, drag, wind, gust intensity and latency) drawn from a distribution $p(\xi)$, and the vehicle evolves as $x_{t+1} = f(x_t, u_t; \xi_\tau)$. A classical controller $u_t = \kappa(x_t, r_t; k)$ tracks a reference r_t with gains k . Learning enters in one of

three ways:

$$(C1, \text{residual}) \quad u_t = \kappa(x_t, r_t; k_0) + \delta_\theta(o_t), \quad (1)$$

$$(C2, \text{gains}) \quad u_t = \kappa(x_t, r_t; k_0 \odot e^z), \quad z \sim \pi_\theta, \quad (2)$$

$$(C3, \text{planner}) \quad v_t = \pi_\theta(o_t), \text{ tracked by } \kappa(\cdot; k_0), \quad (3)$$

where k_0 are nominal gains, o_t the policy’s observation and v_t a velocity command. An adaptation rule U maps the shared parameters θ and experience $\mathcal{D}_\tau$ on a new task to task-specific parameters, $\theta_\tau = U(\theta, \mathcal{D}_\tau)$, and meta-learning chooses θ (and, for some methods, U) to maximise post-adaptation return,

$$\max_{\theta} \mathbb{E}_{\tau \sim p} [J_\tau(U(\theta, \mathcal{D}_\tau))]. \quad (4)$$

For MAML, $U(\theta, \mathcal{D}) = \theta + \alpha \nabla_{\theta} \hat{J}_\tau(\theta; \mathcal{D})$ with a policy-gradient estimate $\hat{J}_\tau$; for PEARL and RL², U is inference in a latent variable or recurrent state, with no gradient step. Non-adapting baselines use $U(\theta, \mathcal{D}) = \theta$. We evaluate J_τ on held-out tasks from p and on OOD tasks beyond its support.

IV. SYSTEM

A. Vehicle model

We simulate many drones at once, each with its own task, in a batched rigid-body model integrated at 500 Hz. Commands are collective thrust and body rates (CTBR), the setpoint PX4 accepts in offboard mode [32]. Each command passes through a latency buffer, then a PX4-like body-rate PI loop that uses the *nominal* inertia, then an X-configuration mixer that gives up yaw torque first when motors saturate. First-order motors (time constant 40 ms) are scaled by per-motor efficiencies. Forces include linear drag (0.3 N s/m), steady wind and Ornstein–Uhlenbeck gusts. The nominal vehicle approximates a Holybro X500 V2 carrying a Jetson and a D435i: mass 2.0 kg, arm 0.25 m and inertia 0.0217/0.0217/0.040 kg m² from PX4’s x500 simulation model; the thrust-to-weight ratio of 2.1, motor lag and drag are our estimates (Sec. VIII).

B. Tasks

A task fixes payload (mass scale; inertia grows by half the relative mass increase, as for a payload near the centre), the efficiency of one randomly chosen motor, drag, steady wind, gust intensity and command latency (Table I). Training and held-out tasks share ranges but use disjoint random seeds. OOD tasks lie beyond every range at once.

TABLE I
TASK DISTRIBUTION. EACH TASK DRAWS EVERY PARAMETER FROM THE GIVEN RANGE.

Parameter	Unit	Train / held-out	OOD
Payload (mass scale)	×	0.85–1.30	1.30–1.40
Weakest motor efficiency	%	75–100	65–75
Drag scale	×	0.5–2.0	2.0–3.0
Steady wind	m/s	0.0–2.5	2.5–4.0
Gust std. (OU, 0.5 s)	m/s	0.0–0.8	0.8–1.5
Command latency	ms	0–30	30–50

C. Classical control

A geometric controller [1] runs at 100 Hz: a PID position loop produces a desired acceleration, limited to 35° tilt, which sets collective thrust and an attitude whose error gives body-rate commands. It has seven gains: proportional, derivative and integral gains for the horizontal and vertical axes, and an attitude gain. The adaptive baseline adds piecewise-constant L1 augmentation [2]: a velocity predictor estimates a lumped acceleration disturbance, whose low-pass-filtered value is subtracted from the desired acceleration.

Tracking. Combinations 1 and 2 are scored on trajectory tracking: each episode follows a smooth random reference (a sum of two cosines per axis, at most 2 m/s horizontally; 15% of episodes are pure hover) for 4 s. The reward is $\exp(-\|e\|/0.25)$ for position error e in metres, minus a small action penalty. An episode ends early if the error exceeds 1.5 m or the tilt exceeds 70°.

Combination 1: residual on the controller. At 50 Hz the residual δ_θ adds a bounded correction to the controller’s output: up to 25% of hover thrust, and up to 1 rad/s in roll and pitch rate (0.5 rad/s in yaw). It observes tracking errors, velocity, attitude, body rates, the reference acceleration, its previous action and the controller’s own command.

Combination 2: controller gains. The policy π_θ is a Gaussian over the logarithm z of the seven gain scales relative to nominal. One gain vector is drawn per episode and the controller flies the whole episode with it.

D. Navigation stack (Combination 3)

Rooms measure 14×8×3 m and contain randomly placed pillars and boxes; a quarter of the boxes hang from the ceiling with their lowest edge at 1.9 m or higher, so the map and planner work in three dimensions. Start and goal sit at opposite ends. The drone is a 0.35 m sphere for collision checks.

Perception and mapping. A depth camera modelled on the D435i casts rays over an 87°×58° field of view up to 5 m, adds noise with standard deviation 0.008 d^2 and drops 2% of pixels. The policy sees a 16×8 image; the mapper uses 48×24. A log-odds voxel map with 0.2 m voxels carves free space along each ray, stopping one voxel short of a hit so that grazing rays don’t erase surfaces.

Planning. A* searches the 26-connected voxel grid inflated by the drone radius plus a 0.15 m margin, replanned at 2 Hz. The carrot is the furthest point up to 1.5 m along the path that is still in straight-line sight.

Local planner. At 10 Hz the local planner outputs a velocity command in the heading frame. The classical controller tracks it through a leashed position setpoint while yaw turns toward the carrot. The learned planner observes the depth image, the carrot and goal in the heading frame, velocity, attitude, altitude and its previous action. It is rewarded for progress along the true shortest path, with penalties for time, proximity, jerky commands and collisions, and a bonus at the goal. The classical local planner pursues the carrot, slows down near depth returns and adds a repulsive term from close returns. For speed, learned planners are trained with a privileged planner: a distance field computed on the true map, equivalent to A* on a perfect map. Evaluation also runs the full online stack.

V. LEARNERS AND BASELINES

All step policies are Gaussian MLPs (64–64 units for tracking, 128–64 for navigation) with a state-independent standard deviation. To keep differences down to the meta-learning rule, the gradient-based methods share one estimator: per-task linear feature baselines with generalised advantage estimation [33] and PPO-clipped updates [34].

MAML family. Each iteration samples 16 tasks and flies 10 episodes per task. One vanilla policy-gradient step with step size 0.1 adapts the policy to each task; all tasks adapt in one batched backward pass. The adapted policies fly 10 new episodes, and a PPO-clipped outer loss on those episodes is backpropagated through the adaptation step. MAML [3] differentiates through the inner gradient (second order); FOMAML drops that term; ANIL adapts only the output layer and the standard deviation [10]; Meta-SGD also learns a step size for every parameter [11]. Like the original MAML for RL, we ignore how the pre-adaptation sampling distribution depends on the parameters [35].

Reptile [9] runs three rounds of per-task PPO (Adam, four epochs per round), each on fresh episodes, then moves the shared parameters toward the mean adapted parameters with a step that anneals from 0.5 to 0.1.

PEARL [12] trains a soft actor-critic [36] conditioned on a five-dimensional latent task variable. A permutation-invariant encoder maps transitions from 32 training tasks to a product-of-Gaussians posterior, trained through the critic loss plus a KL penalty to the prior.

RL² [13], [14] uses a GRU policy with 128 units that sees the previous action, reward and an episode-start flag, and keeps its hidden state over a trial of three episodes on the same task. It is trained with recurrent PPO over whole trials.

Baselines. *Classical* uses nominal gains and no learning; *Classical + L1* adds L1 augmentation. *DR* trains one policy with PPO over the whole task distribution (domain randomisation) and never adapts; *DR + fine-tune* adapts it at test time with the same per-task PPO rounds as Reptile. For Combination 2 the DR baseline is the same Gaussian over gains, trained over all tasks without a meta objective. For navigation we add *end-to-end DR*, a DR planner that sees the goal direction instead of the carrot, with no map or planner.

TABLE II
TRAINING BUDGET IN MILLIONS OF ENVIRONMENT STEPS.

Combination	DR	MAML fam.	Reptile	PEARL	RL ²
C1 residual	9.6	9.6	11.5	1.0	4.3
C2 gains	4.8	6.4	7.7	—	—
C3 planner*	12.0	5.8	7.2	1.0	5.2

*MAML family and Reptile: additional steps after starting from DR.

Training budgets. Table II lists the training budget of each learner. DR and the MAML family saw the same data on tracking, while PEARL and RL² saw less.

Navigation specifics. Training navigation planners from scratch is the most expensive part of the study, so to fit our CPU budget the gradient-based meta-learners start from the trained DR planner and meta-train from there. PEARL and RL² use different networks and train from scratch, so their comparison with the others is confounded by starting point. With the tracking step size (Adam, 3e-3), per-task PPO adaptation wrecked the warm-started navigation planner (Reptile reached 0% success), so navigation uses 3e-4 for both Reptile and DR + fine-tune; the diverged run is kept in the released results.

VI. EXPERIMENTAL PROTOCOL

Tasks and rooms. Every method is scored on fixed task sets: 32 held-out and 32 OOD tasks for tracking, and 24 of each for navigation. Navigation episodes are flown in rooms from a test pool of 128 that is never used in training, each episode in its own room (96 rooms per seed).

Adaptation. Before adapting and after each of three adaptation stages, we fly four deterministic evaluation episodes (the policy mean) per task. A stage of gradient-based adaptation uses 10 exploratory episodes and one update, so three stages use 30 episodes. The MAML family is meta-trained for a single update, so stages 2 and 3 extrapolate beyond the trained procedure; where it matters we also report the one-update result. A PEARL stage adds one episode of context, and an RL² stage is one more episode of the trial. Non-adapting methods are evaluated once.

Full stack. Navigation is scored twice. Learned planners are first scored with the privileged planner, and every method is then scored on the full online stack (depth camera, map built in flight, A*) in 48 unseen rooms per seed, before and after one adaptation stage. The stage’s exploration flights use the privileged planner, like a calibration flight in a known room.

Metrics and statistics. We report tracking RMSE in centimetres, with an early termination counting as 1.5 m error for the rest of the episode, and navigation success: reaching within 0.5 m of the goal before a collision or a 30 s time-out. Learned methods are trained with 2 seeds on the residual problem, 5 on the gain problem and 2 on navigation; intervals are 95% normal intervals over tasks and seeds. Paired comparisons use the same tasks and seeds for both methods, with bootstrap intervals over task–seed pairs. Everything ran on one shared Intel Core i9-9980HK CPU without a GPU.

VII. RESULTS

We organise the results around four questions: does learning beat classical adaptation for tracking (VII-A), where does meta-learning add to it (VII-B), what happens when a learned planner adapts inside the navigation stack (VII-C), and how do methods behave beyond the training distribution (VII-D).

A. Learning versus classical adaptation

A learned residual roughly halves the error of classical adaptation. On held-out tasks the nominal geometric controller tracks with 21.4 cm RMSE and L1 augmentation with 15.5 cm (Table III). A DR residual trained over all tasks reaches 7.3 cm without any adaptation. To make sure L1 is not a weak baseline, we grid-searched its predictor pole and filter bandwidth on training tasks (best: 10 s⁻¹, 40 rad/s); tuning lowers its held-out error only from 14.9 to 14.0 cm on a common set of episodes, still about twice the residual’s.

B. Where meta-learning adds value

On top of a residual, meta-learning adds little. MAML, FOMAML, ANIL and Meta-SGD start slightly worse than DR (8.7, 8.5, 8.6 and 8.7 cm) and improve with 30 adaptation episodes to 7.8, 7.8, 7.7 and 7.8 cm, within the interval of DR’s 7.3 cm (Fig. 2a). With the same training budget as DR (Sec. V), they end where DR starts. Fine-tuning DR with PPO makes it slightly worse (8.0 cm), and Reptile trails at 14.8 cm. Of the context-based methods, PEARL adapts from far less data (10.5 to 9.0 cm with three episodes of context) but does not reach the gradient methods; RL² is the weakest learned residual (17.4 cm).

Adapting the classical gains is where meta-learning pays off. When the policy chooses the controller’s seven gains (Combination 2), every MAML-family method improves markedly with adaptation: Meta-SGD from 15.8 to 10.8 cm, MAML to 12.8 cm and FOMAML to 13.0 cm, while gains tuned by DR stay at 14.5 cm, or 14.3 cm with fine-tuning (Fig. 2b). Paired over the same 160 task–seed combinations, Meta-SGD improves on fine-tuned DR gains by 3.5 cm (95% bootstrap interval 2.8–4.3 cm) and wins on 83% of pairs. Adapted gains remain above the learned residual’s error, but a seven-dimensional gain vector is a far smaller, more interpretable and more easily bounded object to adapt on a real vehicle.

C. Adaptation inside the navigation stack

Gradient-based adaptation degrades learned planners at the meta-trained step size. With the privileged planner, the warm-started MAML-family planners lose success as they adapt: MAML from 73% to 36%, FOMAML from 76% to 31%, ANIL from 76% to 35% and Meta-SGD from 71% to 49% (Table IV, Fig. 2c), with collisions rising as success falls. The decline already appears after the single update they were meta-trained for (MAML: 46%), so it is not only extrapolation. Methods that adapt conservatively stay flat: Reptile (72% to 71%) and DR with fine-tuning (72% to 71%) take small Adam

TABLE III

TRACKING UNDER NEW DYNAMICS. A LEARNED RESIDUAL ROUGHLY HALVES THE ERROR OF CLASSICAL ADAPTATION, AND META-LEARNING HELPS MOST WHEN IT TUNES THE CONTROLLER’S GAINS. HELD-OUT RMSE BEFORE AND AFTER ADAPTING (MEAN, WITH THE 95% HALF-INTERVAL AFTER ADAPTING), OOD RMSE AND OOD CRASH RATE AFTER ADAPTING. DATA: ADAPTATION EPISODES USED. BOLD: BEST IN COLUMN. RESIDUAL LEARNERS USE 2 TRAINING SEEDS, GAIN LEARNERS 5.

Method	Data (ep.)	Combination 1: residual on controller				Combination 2: controller gains		
		Before (cm)	After (cm)	OOD (cm)	OOD crash (%)	Before (cm)	After (cm)	OOD (cm)
<i>Classical</i>								
Classical (nominal gains)	0	21.4	21.4	78.7	34	21.4	21.4	78.7
Classical + L1	0	15.5	15.5	72.1	52	–	–	–
<i>Non-meta learned</i>								
DR (no adapt.)	0	7.3	7.3	63.1	44	14.5	14.5	77.4
DR + fine-tune	30	7.3	8.0±1.0	63.6	43	14.5	14.3±1.2	75.4
<i>Gradient meta-learning</i>								
MAML	30	8.7	7.8±1.1	70.8	49	15.9	12.8±1.4	70.5
FOMAML	30	8.5	7.8±1.0	66.9	46	15.9	13.0±1.4	73.1
ANIL	30	8.6	7.7±1.1	70.4	49	–	–	–
Meta-SGD	30	8.7	7.8±1.1	69.3	48	15.8	10.8±1.1	69.3
Reptile	30	15.2	14.8±2.0	68.6	36	13.6	13.5±1.1	75.8
<i>Context meta-learning</i>								
PEARL	3	10.5	9.0±1.1	69.2	49	–	–	–
RL ²	3	18.4	17.4±1.5	64.6	36	–	–	–

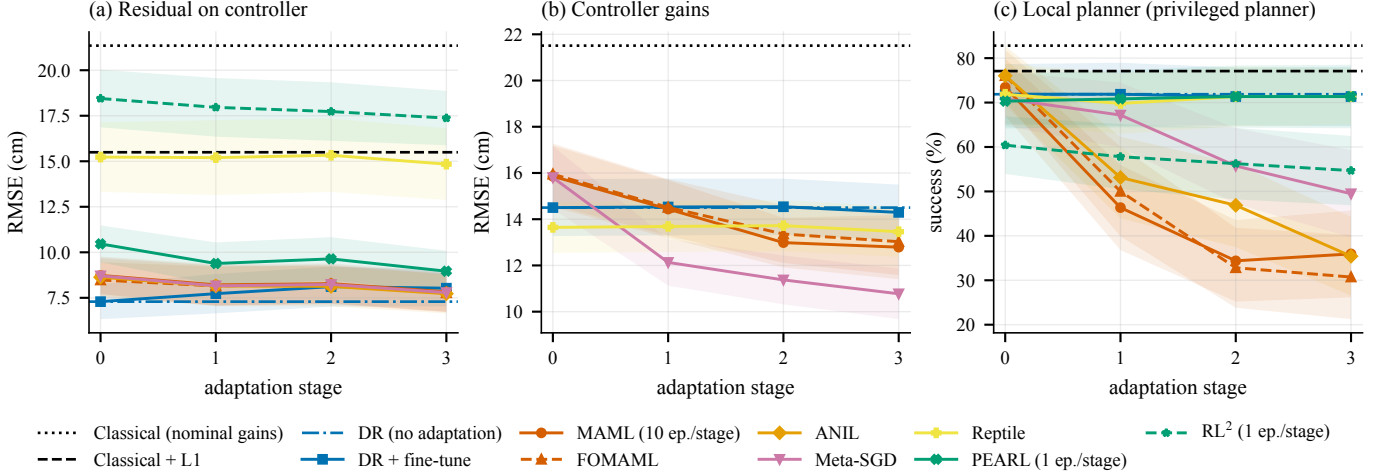


Fig. 2. Adaptation pays off when it tunes classical gains (b), barely moves a learned residual (a), and at the meta-trained step size degrades learned navigation planners (c). Held-out tasks; mean and 95% interval over tasks and seeds; horizontal lines are methods that do not adapt. A stage is one update from 10 episodes for the gradient-based methods and one more episode of experience for PEARL and RL². End-to-end planners are omitted from (c); see Table IV.

steps, and PEARL (70% to 71%) takes none. RL² does not improve with experience (60% to 55%).

The decline is a step-size and data-budget effect. Scaling the inner step at test time (Table V) shows that the drop occurs at the meta-trained step with 10 episodes per update (MAML: 69% to 25%). With a 10× smaller step MAML holds at 72% (Meta-SGD: 70%), and with three times more episodes per update at the original step it holds at 64%. Smaller steps prevent the damage but buy little: success stays within a few points of its pre-adaptation level. Step sizes meta-learned for one task family therefore do not transfer to the sparse, high-variance returns of navigation, and on a real vehicle they should be scaled down or paired with more data per update.

On the full stack, the classical planner is the most

reliable. With a map built in flight and A* replanning, the classical local planner reaches the goal in 73% of 96 flights in unseen rooms with 7% collisions, and L1 augmentation does not change this (73%). The learned planners reach 31–55% with 42–69% collisions. Learned planners also lose far more than the classical planner when the privileged planner is replaced by the online map (DR: 72% to 44%; classical: 83% to 73%). They were trained against carrots from a perfect map, and the online map’s late-discovered obstacles and replanned paths fall outside that training distribution. Fig. 3 shows a recorded classical flight.

The planner helps a learned policy only when the map is good. With the privileged planner, the DR planner that follows the carrot beats the end-to-end DR planner that sees only the

TABLE IV

NAVIGATION IN UNSEEN ROOMS. THE CLASSICAL LOCAL PLANNER IS THE MOST RELIABLE ON THE FULL STACK, AND GRADIENT-BASED ADAPTATION AT THE META-TRAINED STEP SIZE LOWERS LEARNED PLANNERS' SUCCESS. SUCCESS AND COLLISION RATES BEFORE AND AFTER ADAPTING, WITH THE PRIVILEGED PLANNER (3 STAGES) AND ON THE FULL ONLINE-MAPPING STACK (1 STAGE), AND OOD SUCCESS. 2 TRAINING SEEDS; EACH EPISODE IN ITS OWN UNSEEN ROOM. BOLD: BEST IN COLUMN.

Method	Data (ep.)	Privileged planner (96 rooms)			Full stack (48 rooms)			OOD
		Before (%)	After (%)	Coll. (%)	Before (%)	After (%)	Coll. (%)	After (%)
<i>Classical</i>								
Classical (nominal gains)	0	83	83	14	73	73	7	16
Classical + L1	0	77	77	21	73	73	9	8
<i>Non-meta learned</i>								
DR (no adaptation)	0	72	72	28	44	44	56	13
DR + fine-tune	30	72	71	29	44	50	50	17
End-to-end DR	0	52	52	48	51	51	49	10
End-to-end DR + fine-tune	30	52	46	54	51	47	53	16
<i>Gradient meta-learning (warm-started)</i>								
MAML	30	73	36	60	46	32	67	8
FOMAML	30	76	31	69	53	31	69	5
ANIL	30	76	35	65	49	45	55	8
Meta-SGD	30	71	49	50	46	48	51	6
Reptile	30	72	71	29	46	47	53	14
<i>Context meta-learning (from scratch)</i>								
PEARL	3	70	71	26	50	55	42	9
RL ²	3	60	55	45	34	39	60	8

TABLE V

THE NAVIGATION DECLINE IS A STEP-SIZE EFFECT. SUCCESS BY ADAPTATION STAGE AND FINAL COLLISION RATE WITH THE INNER STEP SCALED FROM ITS META-TRAINED VALUE, OR WITH 30 INSTEAD OF 10 EPISODES PER UPDATE. PRIVILEGED PLANNER, 16 HELD-OUT TASKS IN UNSEEN ROOMS, SEED-0 CHECKPOINTS.

Configuration	Success by stage (%)				Coll. (%)
	0	1	2	3	
MAML, meta-trained step	69	58	39	25	69
MAML, step $\times 0.3$	69	75	64	66	34
MAML, step $\times 0.1$	69	72	70	72	28
MAML, 30 ep./stage	69	75	62	64	36
Meta-SGD, learned steps	67	64	50	38	58
Meta-SGD, steps $\times 0.3$	67	69	67	73	27
Meta-SGD, steps $\times 0.1$	67	70	69	70	30

goal direction (72% vs 52%). On the full stack the order flips (44% vs 51%): the end-to-end planner never depended on a map, so it loses nothing when the map is imperfect.

D. Beyond the training distribution

Every method fails out of distribution, and learned residuals crash more. On the OOD tasks, which combine heavier payload, weaker motors, stronger wind and longer latency, no tracking method gets below 63.1 cm RMSE and no navigation method exceeds 17% success. Learned residuals crash in 43–49% of OOD episodes, against 34% for the nominal controller, and three adaptation stages change OOD error by at most 7 cm. The pattern already appears just outside the training ranges: on a milder split that exceeds each range only slightly, the DR residual still tracks better than the nominal controller (41.1 vs 61.5 cm), but DR and MAML-family residuals crash in 14–18% of episodes against 9%. Learned residuals thus trade safety margin for accuracy at

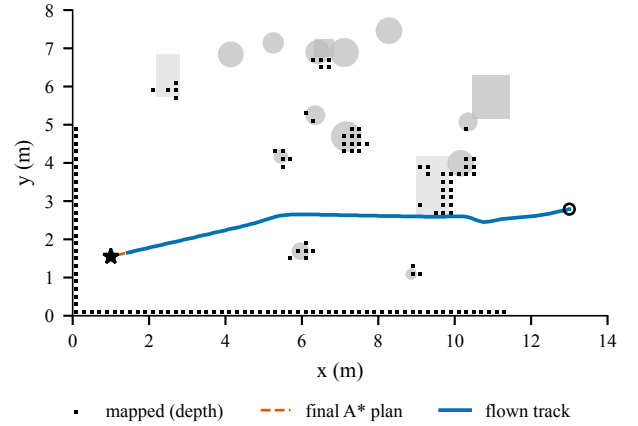


Fig. 3. The classical stack maps an unseen room from its depth camera and reaches the goal. Grey: true obstacles (paler boxes hang overhead and are flown under); black: voxels mapped by the end of the flight; dashed: final A* plan; blue: flown track from start (circle) to goal (star).

the edge of the envelope, and the plain classical controller degrades most gracefully.

VIII. REAL-WORLD PLATFORM AND TEST PLAN

The stack needs a flight controller that accepts CTBR or velocity setpoints from a companion computer at 50 Hz or more, a depth camera, onboard compute for mapping and planning, and a vehicle that is safe in a netted cage. Table VI compares indoor research platforms in the \$1–3k range, from manufacturer pages checked on 2026-10-05.

We recommend a Holybro X500 V2 with a Jetson Orin Nano Super and a RealSense D435i, about \$1.6–2.0k in parts. PX4's offboard mode takes body rates and collective thrust

TABLE VI

CANDIDATE PLATFORMS. ONLY THE X500 BUILD IS IN STOCK, WITHIN BUDGET, AND CARRIES A DEPTH CAMERA THAT MATCHES THE SIMULATOR. STATUS FROM MANUFACTURER STORES ON 2026-10-05.

Platform	Mass	Depth	CTBR	Status
X500 V2 build [†]	≈2.0 kg	D435i	PX4	in stock
Crazyflie 2.1 Brushless	37 g	none	partial	listed
Starling 2	285 g	ToF	PX4	unavailable new
Starling 2 Max	566 g	ToF (C29 only)	PX4	over budget
PX4 Vision V1.5	893 g*	discontinued	PX4	sold out

[†]Holybro X500 V2 + Jetson Orin Nano Super + D435i. *Without battery.

directly, the policies’ own action space. The D435i is the camera the simulator models, and the X500 is PX4’s reference airframe, so its simulator parameters come from a published model. The cost of this choice is a 2 kg vehicle with 10-inch propellers, which must fly inside a netted cage. The Crazyflie is the safest vehicle for checking that low-level control transfers, but it cannot carry a depth camera.

The test plan has five stages, each gated by the previous one: (0) system identification of mass, inertia, thrust curve and motor lag to replace our estimates; (1) hover with the policy in the loop under motion capture; (2) trajectory tracking with known added payloads, compared with the simulator; (3) known foam obstacles with live depth mapping; (4) an unknown room with onboard visual-inertial odometry only. Safety layers include an RC kill switch, PX4 geofence, offboard-loss failsafe, and a watchdog that falls back to hold if commands arrive late.

IX. LIMITATIONS

All results are in simulation; no real flights have been made. The thrust-to-weight ratio, motor lag and drag of the simulated vehicle are estimates. The aerodynamics are simple (linear drag, no ground effect or rotor drag), the depth model lacks texture- and lighting-dependent stereo artefacts, and state estimation is perfect. Compute was a single shared CPU, which limited seeds (2 for the residual, 5 for gains, 2 for navigation) and training length; the step-size study uses one seed. On navigation, the gradient-based learners start from the trained DR planner while PEARL and RL² train from scratch, and PEARL and RL² were trained on fewer environment steps (Sec. V), so their comparison with the others is confounded. Learned planners were trained with a privileged planner and tested on the online stack, which explains part of their full-stack gap. The MAML implementations ignore the dependence of the pre-adaptation sampling distribution on the parameters. Finally, our OOD tasks are harsh enough that every method fails on them.

X. CONCLUSION

We combined seven meta-learning algorithms with a classical quadrotor stack at three insertion points and found that where learning enters matters more than which algorithm is used. A robust learned residual captures most of the tracking benefit without meta-learning. Meta-learning pays off when it adapts the classical controller’s gains. In navigation, gradient-based adaptation at meta-trained step sizes made learned

planners less safe, while a classical planner on an online map remained the most reliable. Every method failed beyond the training distribution, with learned residuals crashing more than the classical controller. These results argue for keeping classical components in the loop, adapting their parameters with conservative updates, and testing adaptation out of distribution before trusting it on a real vehicle. Flight tests on the recommended platform are the next step.

ACKNOWLEDGMENT

The simulator, learning algorithms, experiments, benchmark, figures and table scripts, and the text of this paper were produced with an AI coding assistant (Claude Code, Anthropic) working under the author’s direction. Citation metadata were checked against arXiv and Crossref by script. The author reviewed the results and is responsible for all claims.

REFERENCES

- [1] T. Lee, M. Leok, and N. H. McClamroch, “Geometric tracking control of a quadrotor UAV on SE(3),” in *IEEE Conference on Decision and Control (CDC)*, 2010.
- [2] Z. Wu, S. Cheng, K. A. Ackerman, A. Gahlawat, A. Lakshmanan, P. Zhao, and N. Hovakimyan, “L1 adaptive augmentation for geometric tracking control of quadrotors,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [3] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International Conference on Machine Learning (ICML)*, 2017.
- [4] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-Fly enables rapid learning for agile flight in strong winds,” *Science Robotics*, 2022.
- [5] D. Zhang, A. Loquercio, X. Wu, A. Kumar, J. Malik, and M. W. Mueller, “Learning a single near-hover position controller for vastly different quadcopters,” *arXiv preprint arXiv:2209.09232*, 2022.
- [6] J. Eschmann, D. Albani, and G. Loianno, “RAPTOR: A foundation policy for quadrotor control,” *Science Robotics*, 2025.
- [7] T. Yu, D. Quillen, Z. He, R. Julian, A. Narayan, H. Shively, A. Bellathur, K. Hausman, C. Finn, and S. Levine, “Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning,” *arXiv preprint arXiv:1910.10897*, 2019.
- [8] J. Beck, R. Vuorio, E. Z. Liu, Z. Xiong, L. Zintgraf, C. Finn, and S. Whiteson, “A tutorial on meta-reinforcement learning,” *Foundations and Trends in Machine Learning*, 2025.
- [9] A. Nichol, J. Achiam, and J. Schulman, “On first-order meta-learning algorithms,” *arXiv preprint arXiv:1803.02999*, 2018.
- [10] A. Raghu, M. Raghu, S. Bengio, and O. Vinyals, “Rapid learning or feature reuse? towards understanding the effectiveness of MAML,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [11] Z. Li, F. Zhou, F. Chen, and H. Li, “Meta-SGD: Learning to learn quickly for few-shot learning,” *arXiv preprint arXiv:1707.09835*, 2017.
- [12] K. Rakelly, A. Zhou, D. Quillen, C. Finn, and S. Levine, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *International Conference on Machine Learning (ICML)*, 2019.
- [13] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, “RL²: Fast reinforcement learning via slow reinforcement learning,” *arXiv preprint arXiv:1611.02779*, 2016.
- [14] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, and M. Botvinick, “Learning to reinforcement learn,” *arXiv preprint arXiv:1611.05763*, 2016.
- [15] S. M. Richards, N. Azizan, J.-J. Slotine, and M. Pavone, “Adaptive-control-oriented meta-learning for nonlinear systems,” in *Robotics: Science and Systems (RSS)*, 2021.
- [16] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid motor adaptation for legged robots,” in *Robotics: Science and Systems (RSS)*, 2021.

- [17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [18] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [19] A. Molchanov, T. Chen, W. Hönig, J. A. Preiss, N. Ayanian, and G. S. Sukhatme, "Sim-to-(Multi)-Real: Transfer of low-level robust control policies to multiple quadrotors," *arXiv preprint arXiv:1903.04628*, 2019.
- [20] J. Zhou, D. Cao, X. Y. Wang, and S. Li, "MAVEN: A meta-reinforcement learning framework for varying-dynamics expertise in agile quadrotor maneuvers," *arXiv preprint arXiv:2603.10714*, 2026.
- [21] J. Xing, I. Geles, Y. Song, E. Aljalbout, and D. Scaramuzza, "Multi-task reinforcement learning for quadrotors," *arXiv preprint arXiv:2412.12442*, 2024.
- [22] S. Sönmez, L. Montecchio, S. Martini, M. J. Rutherford, A. Rizzo, M. Stefanovic, and K. P. Valavanis, "Reinforcement learning based prediction of PID controller gains for quadrotor UAVs," *arXiv preprint arXiv:2502.04552*, 2025.
- [23] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [24] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," *arXiv preprint arXiv:1812.06298*, 2018.
- [25] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [26] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [27] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, 2021.
- [28] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, 2023.
- [29] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, "Learning to fly—a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control," *arXiv preprint arXiv:2103.02142*, 2021.
- [30] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight," *Science Robotics*, 2022.
- [31] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, "Safe-Control-Gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics," *IEEE Robotics and Automation Letters*, 2022.
- [32] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
- [33] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.
- [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [35] J. Rothfuss, D. Lee, I. Clavera, T. Asfour, and P. Abbeel, "ProMP: Proximal meta-policy search," *arXiv preprint arXiv:1810.06784*, 2018.
- [36] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, 2018.